



Implementation of a Public Key Infrastructure (PKI)

Mohamed H. EL MOKHTAR

July 2025

Contents

1	Introduction	3
1.1	Context and Motivation	3
1.2	What is PKI?	3
1.3	Objectives	3
2	Requirements Analysis	4
2.1	Use Cases	4
2.2	Security Levels	4
3	Solution Selection	5
3.1	Evaluated Solutions	5
3.2	Chosen Solution	5
3.3	Certificate	6
3.4	Certification Authority (CA)	7
4	Implementation	8
4.1	Our PKI Architecture	8
4.2	AWS Infrastructure	9
4.3	Root CA Setup	9
4.4	Subordinate CA Setup	11
5	Certificate Templates and Enrollment	14
5.1	Template Creation	14
5.2	Enrollment Methods	14
6	Demonstrations and Results	16
7	Risk Analysis and Recommendations	17
8	Documentation and Training	18
9	Conclusion and Future Work	19

Abstract

This report details the design, implementation, and validation of an internal Public Key Infrastructure (PKI) using Microsoft AD CS on AWS EC2 instances. The infrastructure consists of an offline standalone Root CA and an online Enterprise Sub-CA integrated into the smartpki.com domain. We demonstrate certificate issuance via Web Enrollment, MMC and analyze risks and recommendations for production deployment.

Introduction

1.1 Context and Motivation

In a context of increasing digitalization, securing electronic communications has become a strategic priority for organizations and institutions. The deployment of a Public Key Infrastructure (PKI) ensures the authenticity, confidentiality, integrity, and non-repudiation of digital communications and documents

1.2 What is PKI?

PKI is the combination of software, encryption technologies, processes, and services that enable an organization to secure its data, communications, and business transactions. PKI relies on the exchange of digital certificates between authenticated users and trusted resources. You use certificates to secure data and to manage identification credentials from users and computers both within and outside of your organization.

1.3 Objectives

- Design a hierarchical PKI aligned with RFC/ETSI/eIDAS standards.
- Deploy a Root CA and a Sub-CA on AWS.
- Implement certificate issuance methods: Web, MMC, and auto-enrollment.
- Document operational procedures and deliver user training.

Requirements Analysis

2.1 Use Cases

The SMART enterprise has multiple use cases for implementing a Public Key Infrastructure. The organization requires the ability to issue different types of digital certificates, including those for web servers, internal machines, and users. These certificates are essential for enabling HTTPS on internal websites, securing communications, and enforcing strong authentication within the network. In the context of this internship project, the focus is primarily on issuing certificates for computers and web servers. These certificates are used to ensure trusted machine authentication and secure web communication (TLS/SSL) within the enterprise environment.

2.2 Security Levels

In a Public Key Infrastructure (PKI), different security models can be adopted depending on the organization's risk profile and operational needs. One of the most secure and widely recommended architectures is the hierarchical model, where a standalone Root Certification Authority (Root CA) is kept offline, and a subordinate Certification Authority (Sub-CA) operates online. This separation provides strong protection against compromise. The Root CA, being offline, is insulated from network-based attacks, and is only used occasionally to issue or renew the Sub-CA certificate. The Sub-CA, integrated with Active Directory and connected to the network, handles all certificate requests and issuance operations for end-entities. To support this model, critical services such as Certificate Revocation List (CRL) and Authority Information Access (AIA) distribution points must be properly configured and made available via HTTP. These mechanisms allow clients to verify the validity of certificates and ensure trust throughout the PKI. This architecture strengthens the integrity of the smartpki.com domain by minimizing the attack surface and ensuring a secure chain of trust from Root CA to end-users.

Solution Selection

3.1 Evaluated Solutions

At the beginning of the project, I conducted research to identify the most suitable solution for implementing an internal PKI that meets the organization's security needs. This initial phase was essential but not straightforward, as it involved comparing different PKI technologies, each with its own strengths and limitations.

The first solution I explored was EJBCA, an open-source PKI system. It appeared to be relatively easy to set up and did not require heavy infrastructure. However, I encountered several challenges, including a lack of comprehensive documentation and tutorials. The available resources were limited, and practical guidance (such as videos or detailed examples) was scarce. These factors made EJBCA less ideal for a time-constrained internship project.

I then turned to Microsoft Active Directory Certificate Services (ADCS), which quickly stood out as the most appropriate choice. ADCS is widely used in enterprise environments and benefits from rich documentation, community support, and a variety of learning materials—including video tutorials, formal training, and detailed Microsoft Learn modules. Its seamless integration with Windows Server and Active Directory made it particularly well-suited to the architecture of the smartpki.com domain.

Other solutions such as OpenXPki and HashiCorp Vault PKI were briefly considered, but due to limited time and their steeper learning curves, I did not pursue them further.

Ultimately, Microsoft ADCS was selected for its maturity, reliability, and ease of integration—allowing me to focus on designing and deploying a functional, standards-compliant PKI within the internship timeframe.

3.2 Chosen Solution

The selected solution is Microsoft Active Directory Certificate Services (AD CS), a native Windows Server role offering a full-featured PKI platform. It provides certificate issuance,

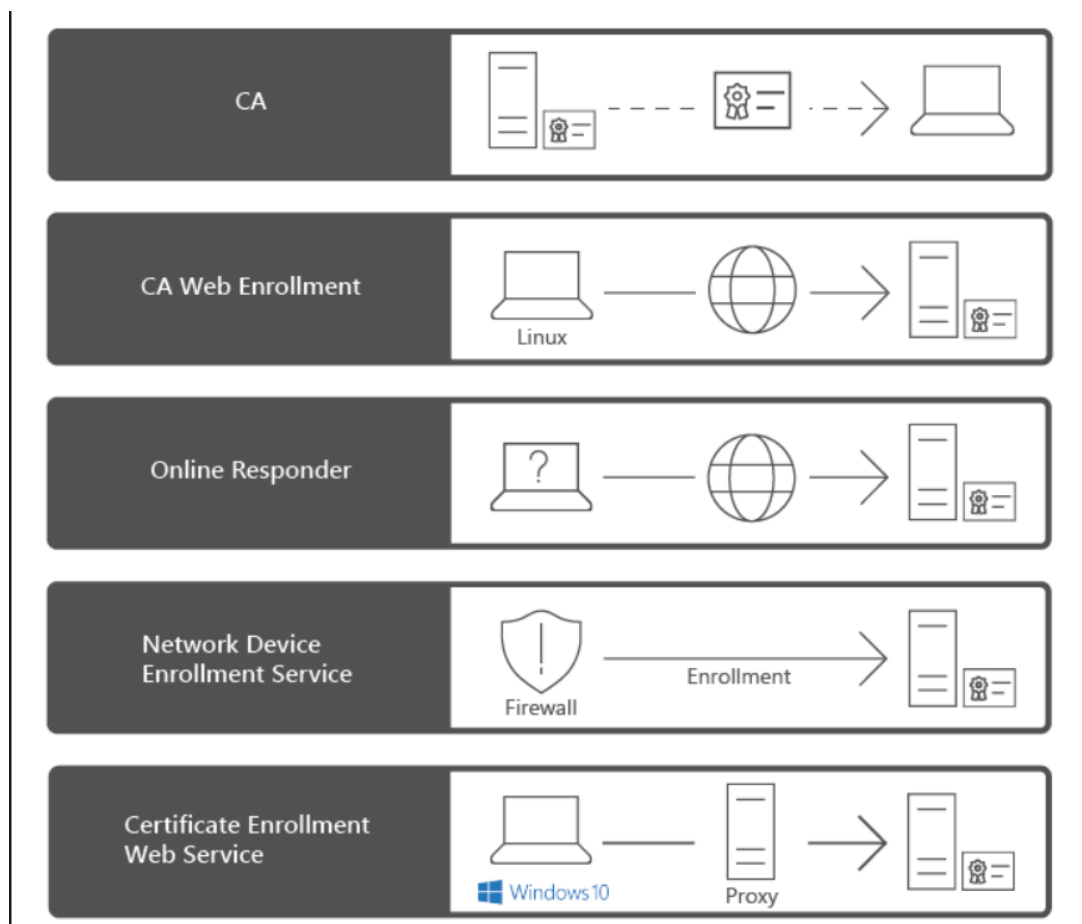
revocation, and management, tightly integrated with Active Directory.

Our architecture uses a two-tier model: an offline standalone Root CA and an online Enterprise Subordinate CA. This setup ensures a high level of security and scalability.

AD CS includes several services, notably:

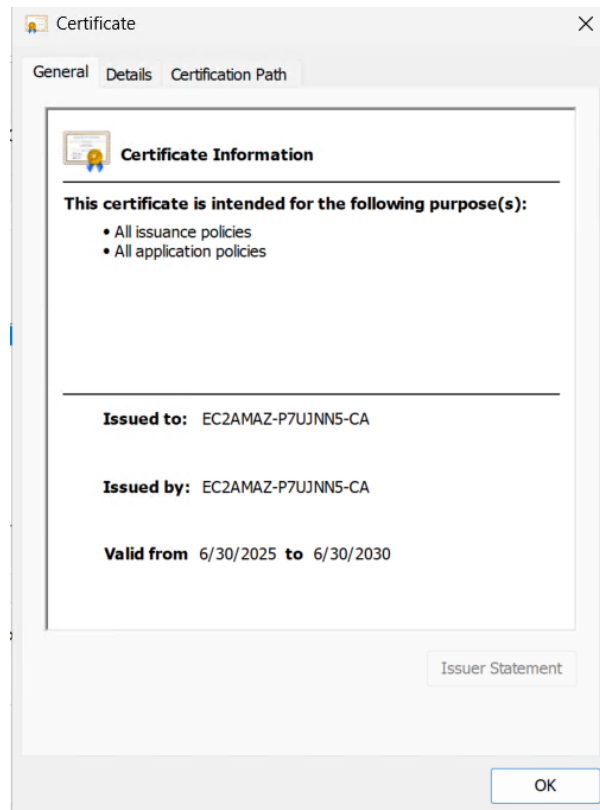
- **Certification Authority (CA)** for issuing and managing certificates.
- **Web Enrollment** for browser-based certificate requests.
- **Certificate Enrollment Services (CES/CEP)** to support domain and non-domain devices.

This solution was chosen for its seamless integration with Windows environments, policy-based management via GPO, and strong community support.



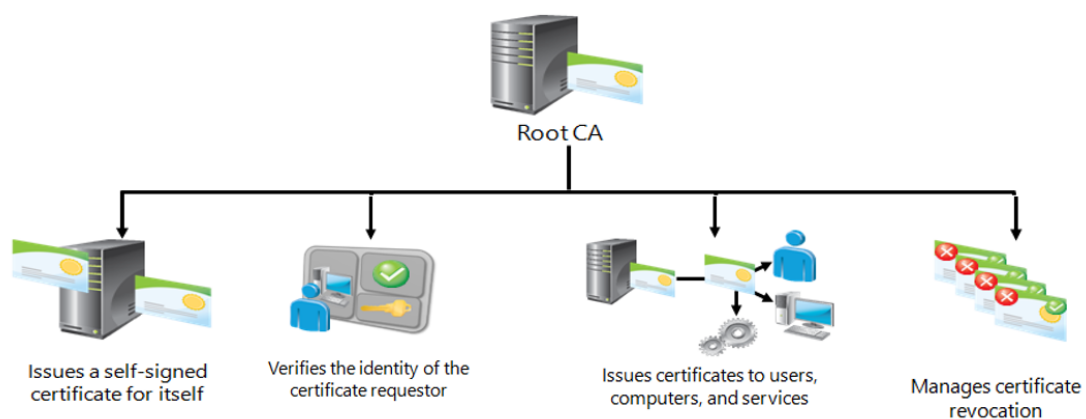
3.3 Certificate

A certificate is a small file that contains several pieces of information about its owner. This data can include the owner's email address, the owner's name, the certificate usage type, the validity period, and the URLs for AIA and CDP locations.



3.4 Certification Authority (CA)

Integrates with Active Directory for certificate publishing and auto-enrollment Role services: Certification Authority (Core issuance/revocation) Web Enrollment Online Responder Network Device Enrollment Service(NDES) Enrollment Policy Web services



Implementation

4.1 Our PKI Architecture

The implemented PKI follows a hierarchical architecture designed to maximize security and scalability. This structure is widely regarded as a best practice in enterprise environments.

Standalone Root CA:

- Maintained offline in a secure environment to protect the integrity of the root keys.
- Used solely to create and sign the certificate of the Subordinate CA.
- Never connected to the network, thereby reducing the attack surface.

Enterprise Subordinate CA:

- Deployed on a domain-joined EC2 instance in the `smartpki.com` Active Directory domain.
- Issues certificates to end-entities such as servers, computers, and users.
- Hosts Web Enrollment services to facilitate manual certificate requests through a browser.

CRL Distribution Points (CDP) & Authority Information Access (AIA):

- CRLs and AIA information are published via HTTPS at `https://pki-sub-ca.smartpki.com/certsrv`.
- These URLs are embedded within each certificate to enable revocation checks and ensure certificate validity.

4.2 AWS Infrastructure

To implement the PKI solution, the project was deployed on an AWS account using foundational cloud networking components. The infrastructure was built within a dedicated Virtual Private Cloud (VPC) named **smartpki-vpc**, which served as the secure and isolated environment for all resources related to the PKI.

VPC and Subnets

A well-structured IP addressing scheme was used:

- **Public Subnet (10.0.1.0/24)**: Hosts the Enterprise Subordinate CA and client machines. This subnet has internet access for updates and external communication.
- **Private Subnet (10.0.0.0/24)**: Dedicated to the Standalone Root CA, kept offline for security purposes and not accessible from the internet.

Security Groups

Security groups were configured to enforce network access control:

- **RootCA-SG**: Restricts all inbound traffic and allows access only from trusted IP addresses for secure administrative operations.
- **Domain-SG**: Permits essential protocols such as HTTP, LDAP, RPC, and SMB to allow communication between domain-joined devices and the CA.

Routing

The default route table was configured with:

- A local route for all intra-VPC communication.
- Internet Gateway attachment only for the public subnet.

This network design ensures both accessibility and security, following best practices for deploying a secure PKI infrastructure on AWS.

4.3 Root CA Setup

To deploy the Root Certificate Authority (CA), an EC2 instance named **PKI-Root-CA** was launched. This instance remains isolated in a private subnet with no internet connectivity to ensure it remains offline, thereby enhancing security.

The setup process involved the following key steps:

- **Access:** Connected to the instance via Remote Desktop Protocol (RDP) from a local machine within the same VPC.
- **Role Installation:** Using Server Manager, the Active Directory Certificate Services (AD CS) role was added:

Server Manager → Add Roles and Features → Server Roles → **Active Directory Certificate Services**.

- **Configuration:** During the configuration phase, the following options were selected:
 - **Setup Type:** Standalone CA.
 - **CA Type:** Root CA.
 - **Cryptographic Provider:** RSA – a public-key encryption algorithm known for its strength and widespread use. RSA uses a key pair and supports key sizes ranging from 1,024 to 4,096 bits.
 - **Key Length:** 2048 bits – sufficient for internal PKI usage.
 - **Common Name:** The CA was assigned a meaningful common name.
 - **Validity Period:** A multi-year period was chosen, as root certificates typically have a long lifespan.
 - **Database Location:** The default or custom path was selected for certificate database and log storage.
- **Result:** Once the setup was completed, the Root CA generated its own self-signed certificate, establishing the trust anchor for the PKI.

Figure 4.1 shows the resulting Root CA certificate:

You can also explore the certificate's details by opening its *Details* tab. There, you will find key information such as:

- **Version:** V3
- **Validity Period:** From the issue date to its expiration
- **Issuer:** The CA itself (self-signed)
- **Signature Algorithm:** RSA with SHA256

In the *Certification Path* tab, only one element appears — the Root CA itself — which confirms that this is a self-signed certificate and the root of trust in the PKI hierarchy.

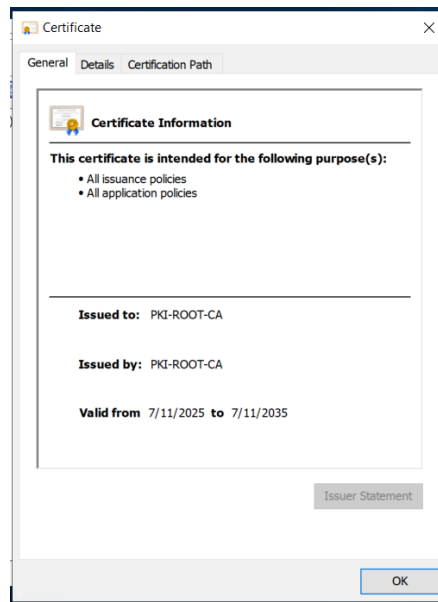


Figure 4.1: Root CA Certificate

4.4 Subordinate CA Setup

The Subordinate CA (Sub-CA), named `pki-sub-ca`, is the central component of the PKI, responsible for issuing certificates to domain members. This server not only acts as an Enterprise Subordinate CA but also hosts the Active Directory Domain Services (AD DS) for the internal domain `smartpki.com`.

Instance Provisioning

An EC2 instance was launched in the public subnet and configured with:

- A static private IPv4 address
- A security group allowing inbound access on essential ports such as HTTP (80), LDAP (389), and RPC (135/445)

The instance was renamed `pki-sub-ca`, and its network configuration was modified to use itself as the primary DNS server.

AD DS Installation and Domain Creation

Instead of joining an existing domain, the server was used to **create a new Active Directory domain** named `smartpki.com` by:

- Installing the Active Directory Domain Services (AD DS) role
- Promoting the machine to a Domain Controller

- Configuring it as the first Domain Controller in a new forest

As a result, `pki-sub-ca` became both the Domain Controller and DNS server for `smartpki.com`.

AD CS Role Installation (Enterprise Subordinate CA)

Once the domain was created, the Active Directory Certificate Services (AD CS) role was installed with the following configuration:

- **CA Type:** Enterprise Subordinate Certification Authority
- **Cryptography:** New RSA private key (2048-bit)
- A Certificate Signing Request (CSR), named `SubCA.req`, was generated to be signed by the offline Root CA

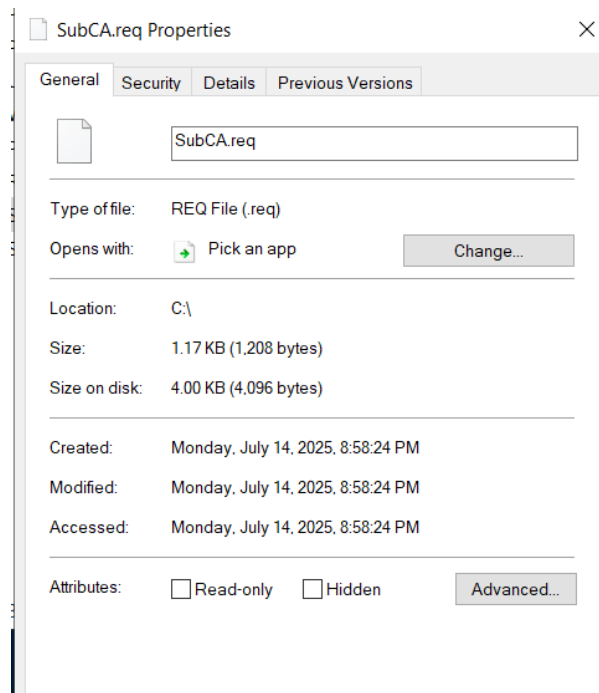


Figure 4.2: CSR Generation for the Subordinate CA

CSR Signing and CA Chain Installation

The CSR was securely transferred to the offline Root CA for signing. After obtaining the signed certificate:

- The Sub-CA certificate and Root CA chain were imported on `pki-sub-ca`
- This completed the installation and activation of the Subordinate CA role

CDP and AIA Configuration

To enable certificate validation and revocation checking:

- Internet Information Services (IIS) was installed on `pki-sub-ca`
- The CRL and AIA distribution URLs were configured and published at: `https://pki-sub-ca.smartpki.com/certsrv`
- These URLs were embedded automatically in each certificate issued by the Subordinate CA

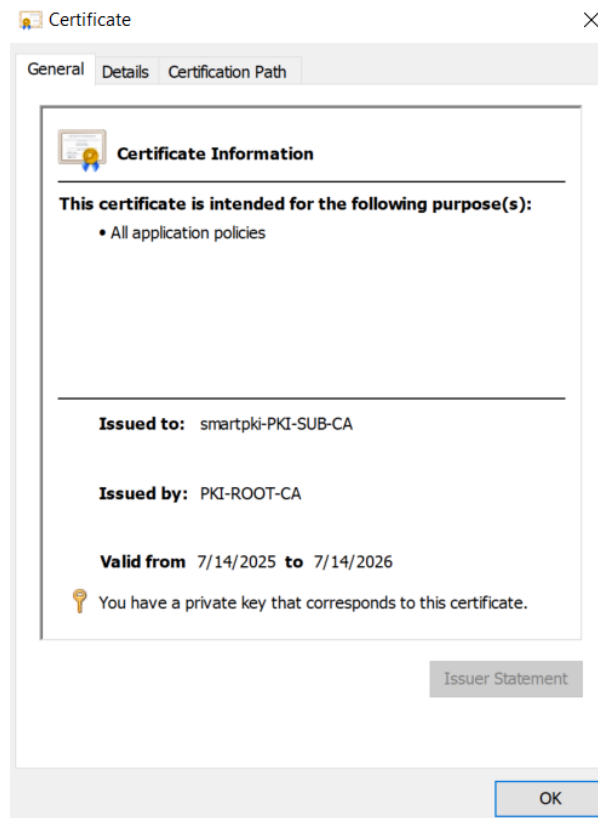


Figure 4.3: Issued Subordinate CA Certificate

In conclusion, `pki-sub-ca` acts both as the domain controller for `smartpki.com` and as the Enterprise Subordinate Certification Authority in the PKI hierarchy.

Certificate Templates and Enrollment

5.1 Template Creation

Certificate templates are predefined models used by an Enterprise Certification Authority (CA) to issue certificates in a consistent and standardized way. They define certificate purpose, validity period, key usage, and issuance requirements (e.g., user authentication, server authentication, file encryption).

Once the Certificate Revocation List (CRL) and the CA infrastructure are operational, certificate enrollment becomes possible through Web Enrollment or the Microsoft Management Console (MMC).

It is important to note that certificate templates are only available when using an **Enterprise CA**. In contrast, a Standalone CA requires each certificate request to be manually created and must include all required certificate details without the use of templates. Certificate templates are stored in Active Directory and retrieved by the CA when a request is made.

For example, templates can be configured to:

- Issue certificates for secure web servers (TLS/SSL)
- Allow users to digitally sign and encrypt emails (S/MIME)
- Authenticate machines in a domain

5.2 Enrollment Methods

Multiple enrollment methods exist in Microsoft PKI:

- Web Enrollment
- MMC (Manual Certificate Request)

- Auto-Enrollment via Group Policy
- Enrollment Web Services (CEP/CES)

Due to time constraints, I focused primarily on Web Enrollment and MMC-based manual enrollment.

Web Enrollment

Web Enrollment allows users and administrators to request certificates via a web interface. On the `smart-client-01` machine, I visited the URL: `https://pki-sub-ca.smartpki.com/certsrv` to:

- Install the CA certificate chain
- Request new certificates for Web Server and User Authentication

However, Web Enrollment only displays Version 1 templates by default, which limits advanced customization. Version 2 and 3 templates—such as those manually created in the Certification Authority console—are not visible through the Web interface.

MMC Enrollment

Using the Certificates snap-in for the Current User or Local Computer in MMC, I was able to:

- View available templates published by the CA
- Request certificates using more flexible and secure templates (e.g., Computer, User, WebServer)
- Import the Root CA and Subordinate CA certificates to the Trusted Root and Intermediate Certificate Authorities stores

This method allows access to advanced templates not supported by the web interface and is better suited for internal environments with domain membership.

Demonstrations and Results

During my internship, I demonstrated the functioning of the PKI solution to my supervisor. These demonstrations included:

- Creating a Root CA and a Subordinate CA
- Setting up the Active Directory Domain `smartpki.com`
- Issuing certificates through Web Enrollment and MMC
- Configuring CRL and AIA distribution over HTTPS

Despite some limitations—such as time constraints and limited experience with Active Directory—I successfully implemented a functional prototype of a PKI environment using Microsoft ADCS on AWS EC2.

Risk Analysis and Recommendations

While a full risk assessment was outside the scope of this internship, several important security considerations were identified:

- **Private Key Protection:** The Root CA should be kept offline to reduce exposure and stored securely.
- **High Availability:** Consider deploying Online Certificate Status Protocol (OCSP) responders and redundant Sub-CAs.
- **Monitoring:** Certificate issuance, expiration, and revocation should be monitored and audited.
- **HSM Integration:** For production environments, a Hardware Security Module (HSM) should be used to protect CA private keys.

AWS security groups were used to restrict traffic to the CAs, but in a real-world scenario, further hardening and auditing would be necessary.

Documentation and Training

Throughout the project, I relied on the following key resources:

- Online training course: *Cryptography: Learn PKI from Scratch* (Udemy)
- Microsoft Learn official documentation: <https://learn.microsoft.com/en-us/training/modules/implement-manage-active-directory-certificate-services>
- Assistance and guidance from ChatGPT

I also documented step-by-step procedures and configurations to support future maintenance and handover.

Conclusion and Future Work

This internship gave me a valuable introduction to Public Key Infrastructure and enterprise IT security practices. I learned how to deploy Microsoft ADACS on cloud infrastructure, configure secure certificate issuance, and integrate PKI with Active Directory.

However, I also realized the importance of having a solid foundation in Active Directory and Windows Server administration—knowledge that is essential for fully understanding and managing PKI systems.

In the future, the following improvements can be made:

- Implement OCSP and configure a CRL distribution point on a dedicated server
- Protect the Root CA with a Hardware Security Module (HSM)
- Explore cross-forest or external trust PKI configurations

Finally, I want to thank the team for their support and patience. I gave this project my full effort and learned a great deal that will benefit my career in cybersecurity and IT infrastructure.